Object Classification, Detection and State Estimation using YOLO-v3 Deep Neural Network and Sensor Fusion of Stereo Camera and LiDAR

Kamalkumar Mehta
Supervisor: Dr. Kamesh Subbarao

Aerospace System Laboratory
Department of Mechanical and Aerospace Engineering
The University of Texas at Arlington

Master Dissertation, June 2021

Table of Contents

1. Motivation & Background
2. YOLO-v3 Architecture
3. Supervised Learning
4. Hyper-parameters Tuning
5. YOLO Algorithm
6. Training Results
7. Optical Sensor & Calibration
8. Sensor Fusion
9. Simulation Results
10. Experimental Results
11. Conclusion & Future Work

Table of Contents

1. Motivation & Background
2. YOLO-v3 Architecture
3. Supervised Learning
4. Hyper-parameters Tuning
5. YOLO Algorithm
6. Training Results
7. Optical Sensor & Calibration
8. Sensor Fusion
9. Simulation Results
10. Experimental Results
11. Conclusion & Future Work

Motivation

- Real-time object detection framework plays essential part in an autonomous drivers assistance system (ADAS) to avoid collision with other vehicles.
- It has wide use in many autonomous tasks such collision avoidance in self-driving vehicle, simultaneous localization and mapping (SLAM) in a mobile robot, personal security, surveillance, target tracking, and identity verification.
- The other object detection frameworks have few drawbacks which limits the use in real-time object detection application.
- Whereas, YOLO is the unified approach to objects detection which requires single regression training and have faster inference time.

Background

- The region proposal based object detection framework (R-CNN) was introduced by Girshick et al. in 2014 which integrates AlexNet and region based selective search. [1]
- To overcome the drawbacks of R-CNN, improved versions Fast-RCNN and Faster-RCNN has been proposed by Girshick et al. in 2015 and Ren et al. in 2016 respectively. [2]
- Redmon et al. in 2016 introduced the unified object detection framework (YOLO) that passes the input image once to detect objects. [3]
- Further improvement done by Redmon and Farhadi and proposed YOLO-v3 in 2018 in order to increase accuracy in detection. [4]

---

[1] Ross Girshick et al. *Rich feature hierarchies for accurate object detection and semantic segmentation*. 2014. arXiv: 1311.2524 [cs.CV]

[2] Shaoqing Ren et al. *Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks*. 2016. arXiv: 1506.01497 [cs.CV]

[3] Joseph Redmon et al. "You only look once: Unified, real-time object detection". In: *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2016, pp. 779–788

[4] Joseph Redmon and Ali Farhadi. "Yolov3: An incremental improvement". In: *arXiv preprint arXiv:1804.02767* (2018)

Data Pipeline

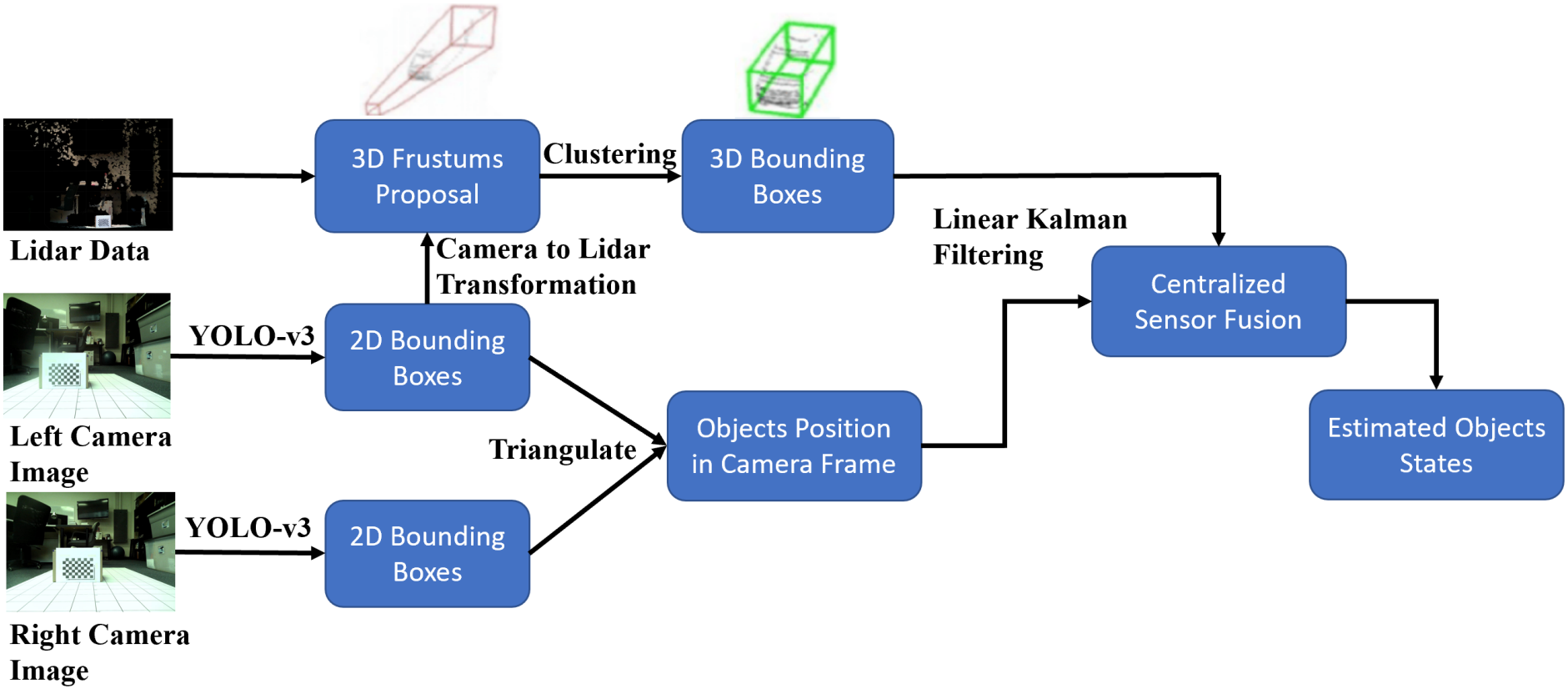


Table of Contents

1. Motivation & Background
2. YOLO-v3 Architecture
3. Supervised Learning
4. Hyper-parameters Tuning
5. YOLO Algorithm
6. Training Results
7. Optical Sensor & Calibration
8. Sensor Fusion
9. Simulation Results
10. Experimental Results
11. Conclusion & Future Work

YOLO-v3 Architecture

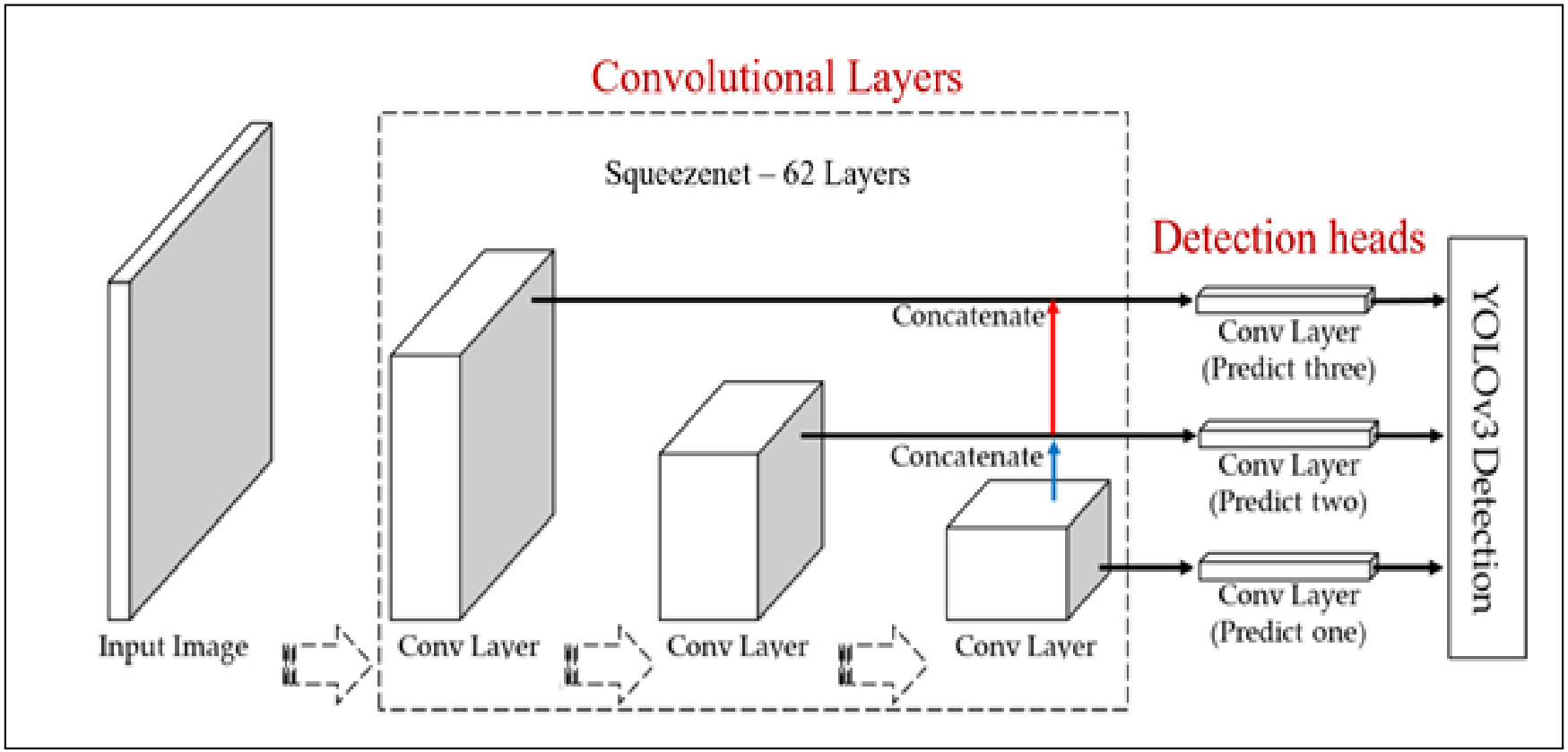


Figure: Stacks of Layers in YOLO-v3 Network [5]

[5] Haojie Ma et al. "Detection of Collapsed Buildings in Post-Earthquake Remote Sensing Images Based on the Improved YOLOv3". In: *Remote Sensing* 12.1 (2020). ISSN: 2072-4292. DOI: 10.3390/rs12010044

Layers in Neural Network Architecture

Input Layer

- Filters the Input Image which size does not compatible to network

Convolutional Layer

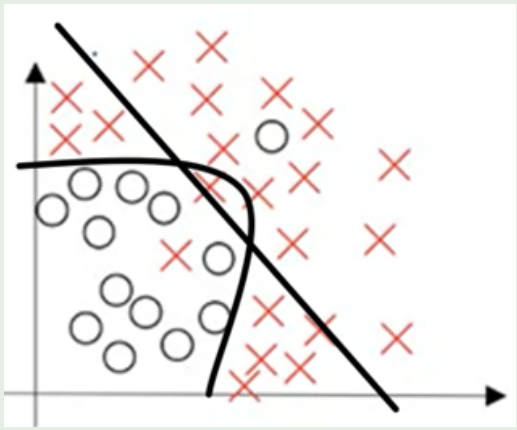
- Performs convolutional operation to detects features from the image e.g. edges, textures, patterns, and parts
- Convolutional operation reduces the size of input image
- The size of filter is pre-define based on the level of feature needs to detect
- Each filter learns one feature of the image

Non-linear Activation Layer

- Impose low-level non-linearity in feature map generated by Conv. layer.
- Reduce the bias in overall training error.

Example

Non-linear Activation



Layers in Neural Network Architecture

Pooling or Down-sampling Layer

- Extracts the most significant number from image by pre-defined size of window
- Reduce computational load, increase speed in training, and prevent over-fitting

Fully Connected Layer

- To combine all the features detected by the Conv. layer and generate classification score for each class
- All the nodes of fully-connected layer are connected with each element of the Conv. layer
- Because of too many connections, learning process slows down

Up-sampling Layer

- Recover the original Image by increase the resolution of activation
- "Skip-connection" is efficient way to recover the high-resolution of image
- Mainly use in task when high resolution is essential e.g. object localization & detection
- Slows down the processing time

Table of Contents

1. Motivation & Background
2. YOLO-v3 Architecture
3. Supervised Learning
4. Hyper-parameters Tuning
5. YOLO Algorithm
6. Training Results
7. Optical Sensor & Calibration
8. Sensor Fusion
9. Simulation Results
10. Experimental Results
11. Conclusion & Future Work

Data Preprocessing

Capture Video
Time: 3 min 22 sec

Grab all frames
Frames: 9.1k

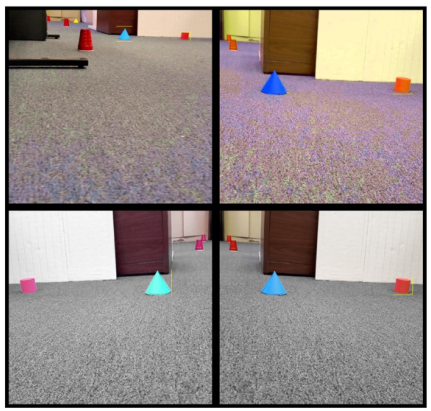
Create labelled dataset

Format:

Image_Location	Class_names …
"C:\...\im.jpg	[100 20 30 60]

Training 98%

Testing 2%



Data Augmentation

Improve the accuracy
Randomly transforming the data during training

Color jitter
Random horizontal flip
Random scaling by 10%

Stochastic Gradient Descent with Momentum (SGDM)

- Neural network impose has non-linearity that causes the most of the loss function to become non-convex
- Iterative gradient descent based optimization of appropriate convex function with proper parameter initialization leads to the global minima.

Loss Function

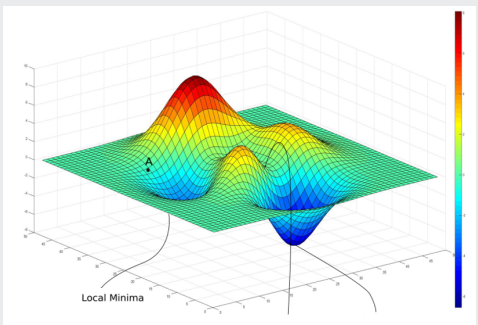


Figure: Non-convex [6]

$$\mathcal{L}(\hat{y}, y) = \frac{1}{2}(\hat{y} - y)^2$$

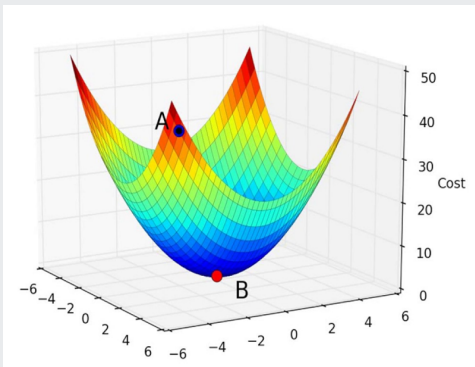


Figure: Convex [7]

$$\mathcal{L}(\hat{y}, y) = -(y \log \hat{y} + (1 - y) \log(1 - \hat{y}))$$

[6] K. Kolev. "Convexity in Image-Based 3D Surface Reconstruction". In: 2012

[7] K. Kolev. "Convexity in Image-Based 3D Surface Reconstruction". In: 2012

Stochastic Gradient Descent with Momentum (SGDM)

Cost Function

- Loss function is define for one training example, the extension of loss function for multiple training example is called cost function

$$\mathcal{J}(\omega, b) = \frac{1}{m}\sum_{i=1}^{m} L\left(\hat{y}^{(i)}, y^{(i)}\right) = -\frac{1}{m}\sum_{i=1}^{m}[y^{(i)} \log \hat{y}^{(i)} + \left(1 - y^{(i)}\right) \log \left(1 - \hat{y}^{(i)}\right)]$$

Momentum

- SGDM is combination of the conventional gradient descent algorithm uses with momentum term
- Momentum is defined by exponentially weighted average of training parameters as follows

$$V_{dW} = \beta v_{dW} + (1 - \beta)dW$$
$$V_{db} = \beta v_{db} + (1 - \beta)db$$

Stochastic Gradient Descent with Momentum (SGDM)

Parameter updates

- The momentum parameter β represents how quick the gradient needs to move towards global minima and it should be between 0 and 1
- Initial Bias correction: $\frac{V_{dW}}{1-\beta^t}$
- Momentum term damping out the oscillations by averaging the positive and negative values in vertical direction
- The training parameters updates after calculating momentum:

$$W = W - \alpha \cdot V_{dW}$$
$$b = b - \alpha \cdot V_{db}$$

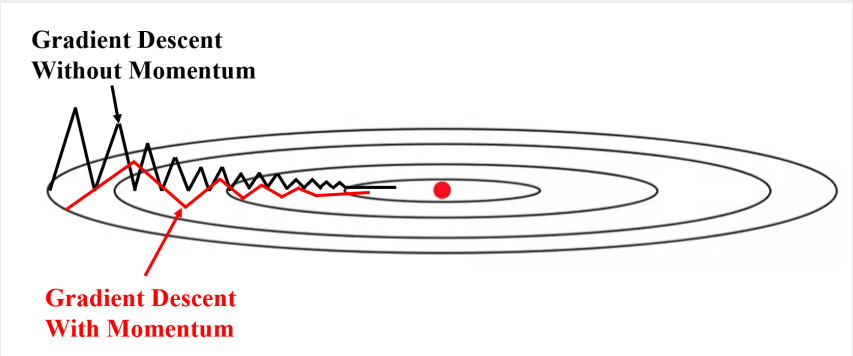


Figure: Robustness of Stochastic Gradient Descent with Momentum

Stochastic Gradient Descent with Momentum (SGDM)

Forward and Backward Propagation

- **Backward Propagation:** Fine tune the training parameters based on the error from previous iteration
- **Forward Propagation:** Derive the amount of error after tuning of the training parameters

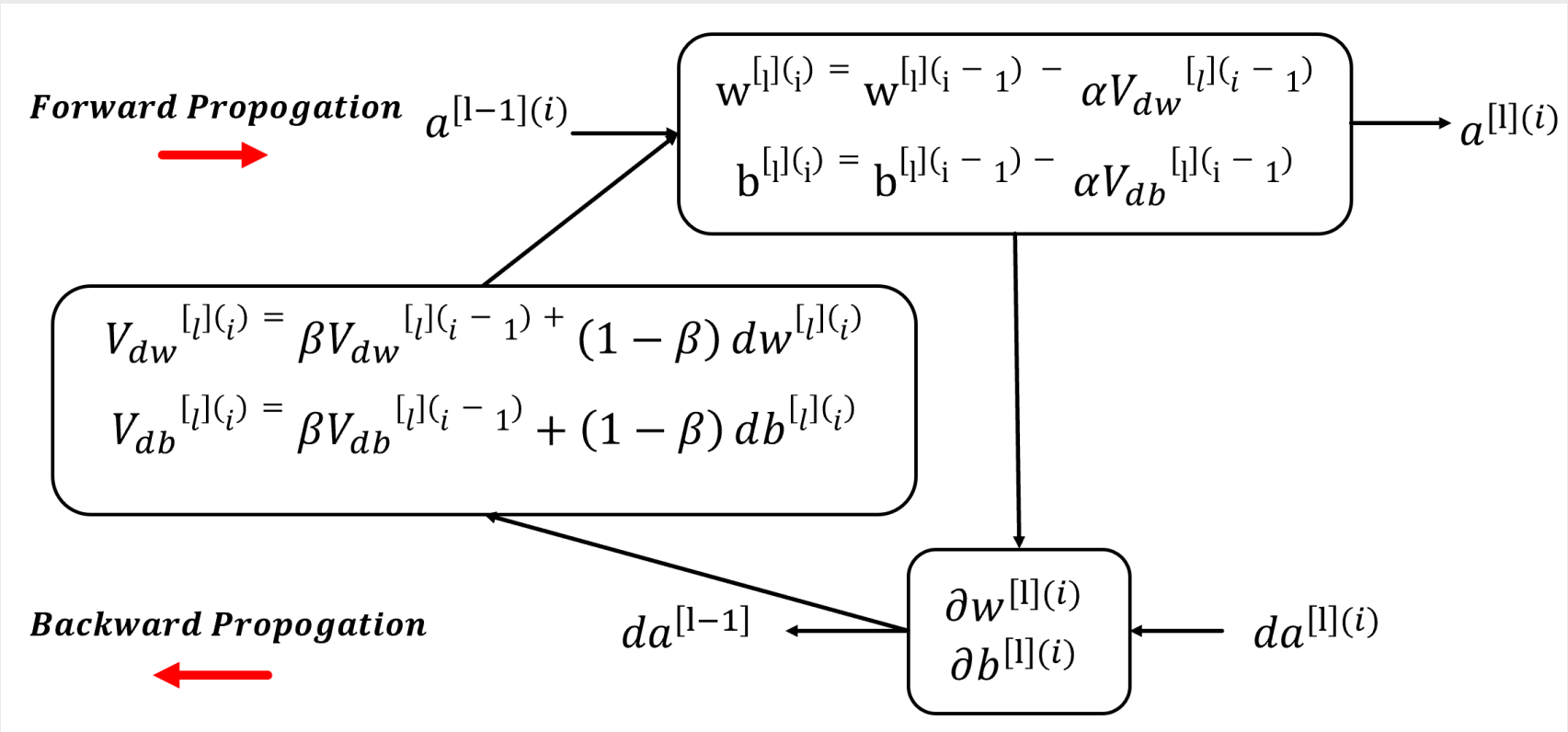


Table of Contents

1. Motivation & Background
2. YOLO-v3 Architecture
3. Supervised Learning
4. Hyper-parameters Tuning
5. YOLO Algorithm
6. Training Results
7. Optical Sensor & Calibration
8. Sensor Fusion
9. Simulation Results
10. Experimental Results
11. Conclusion & Future Work

Bias - Variance Trade-off

- **Bias:** Error due to deviation in estimation from true value
- **Variance:** Error due to deviation in measurement from expectation
- Bias and Variance are inversely proportional to each other

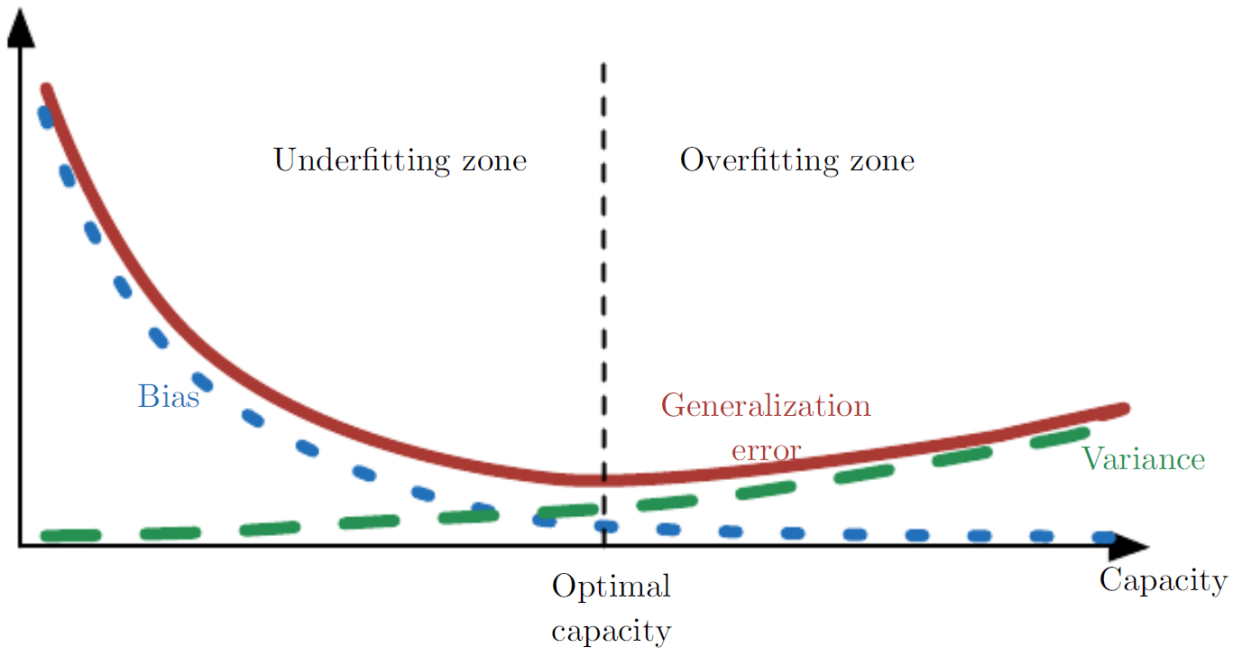


Figure: Relationship between Bias and Variance [8]

[8] Ian Goodfellow et al. *Deep learning*. Vol. 1. 2. MIT press Cambridge, 2016

Hyper-parameters Tuning

L2 Regularization

- It prevents the network from over-fitting on outliers
- It penalize the weight matrix to prevent it from being too large
- The higher value of λ is expected; however, larger than optimal value tends increase the variance

Cost Function: $J(w, b) = \frac{1}{m} \sum_{i=1}^{m} L\left(a^{(i)}, y\right) + \frac{\lambda}{2m} \|w\|_2^2$

Mini-batch Size

- Large training dataset divided into small batches to improve the training speed
- Small size mini-batch training is extremely noisy and larger size tends to slower the convergence time
- Optimal size of mini-batch should be selected to balance between noise and speed

Hyper-parameters Tuning

Number of Epoch

- It represents the numbers of time the complete dataset passes through the network during training
- It should be large enough that allows an algorithm to run until sufficient accuracy is achieved.

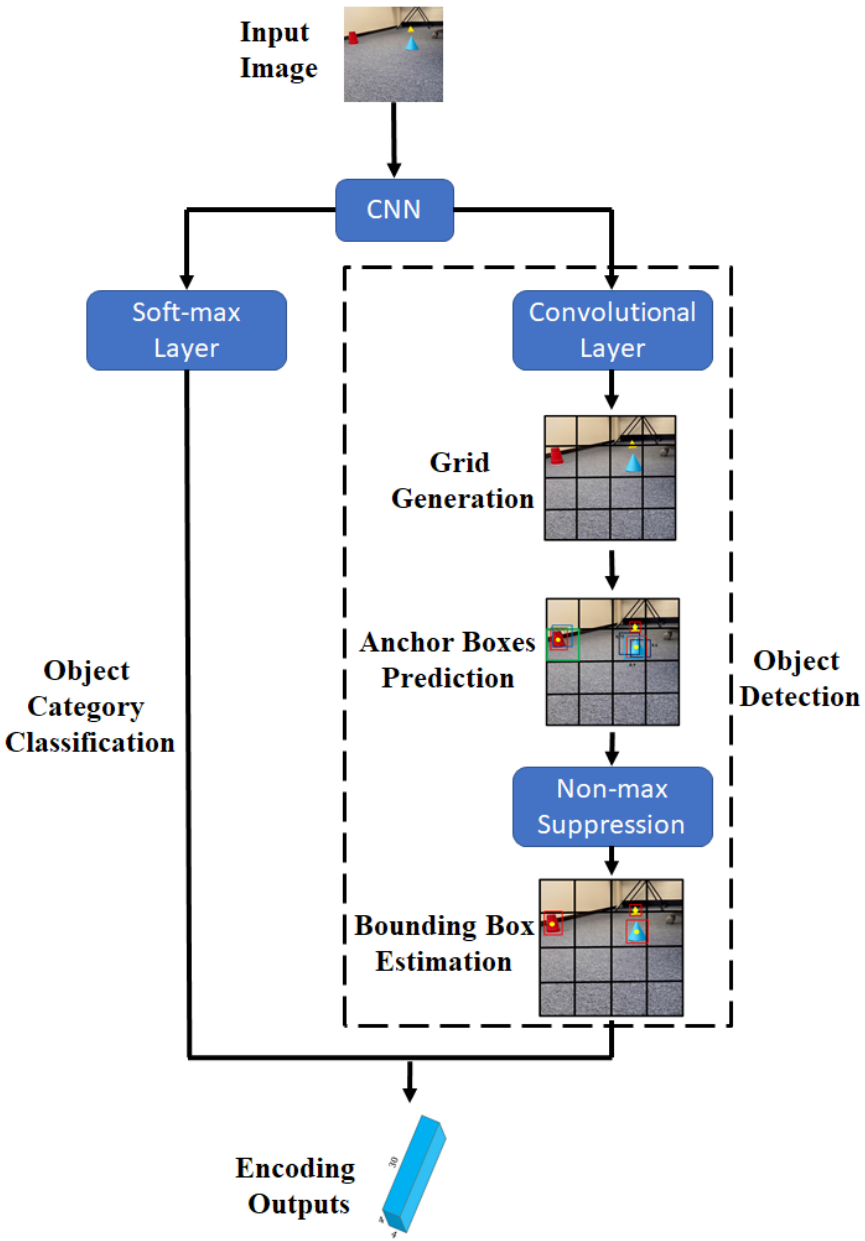
Learning Rate

- It represents the forward step size of gradient
- The variation in learning rate is adopted as the training progresses to perform training on complex dataset
- **Warming-up**- Rate is initially low and increases exponentially till the preset value
- **Learning Rate Decay**- At the final stage, rate reduces gradually or in discrete steps

Table of Contents

1. Motivation & Background
2. YOLO-v3 Architecture
3. Supervised Learning
4. Hyper-parameters Tuning
5. YOLO Algorithm
6. Training Results
7. Optical Sensor & Calibration
8. Sensor Fusion
9. Simulation Results
10. Experimental Results
11. Conclusion & Future Work

YOLO Algorithm



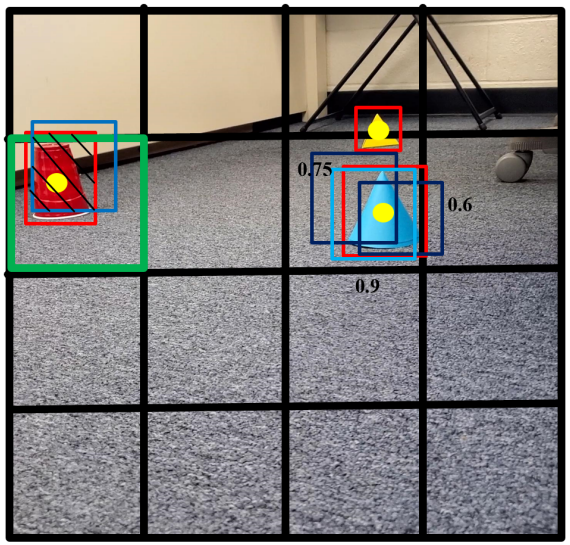


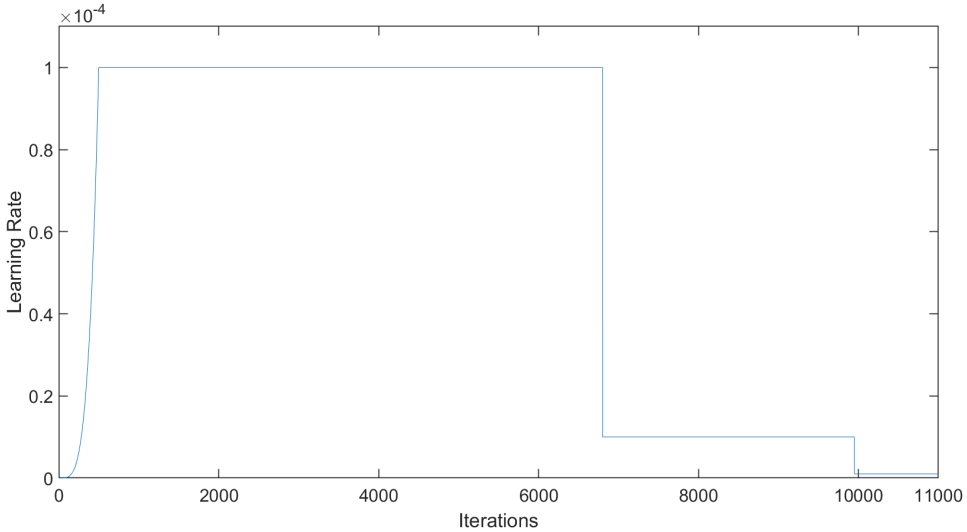
Table of Contents

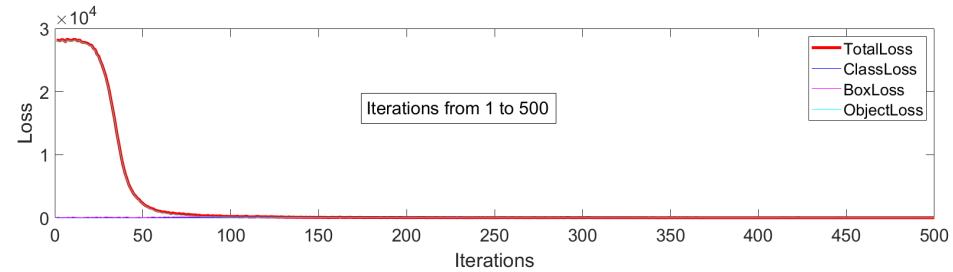
1. Motivation & Background
2. YOLO-v3 Architecture
3. Supervised Learning
4. Hyper-parameters Tuning
5. YOLO Algorithm
6. Training Results
7. Optical Sensor & Calibration
8. Sensor Fusion
9. Simulation Results
10. Experimental Results
11. Conclusion & Future Work

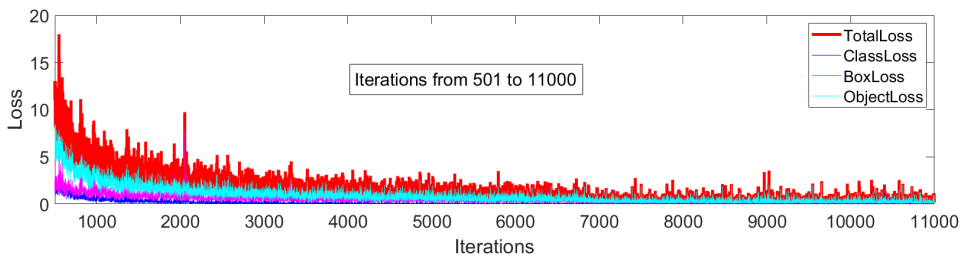
Optimal Value of Hyper-parameters

Hyperparameter Name	**Optimal Value**
Number of Iterations	11000
Total Number of Data	10989
Learning Rate	0.0001
Warm-up Period	500 iterations
L2 Regularization Parameter	0.0005
Mini-batch Size	10
Train batch	98%
Test batch	2%
Mean IoU of Anchor Box Estimation	0.8738
Number of Anchor Boxes per Detection Head	12

Learning Rate & Loss Plot







Precision Plot

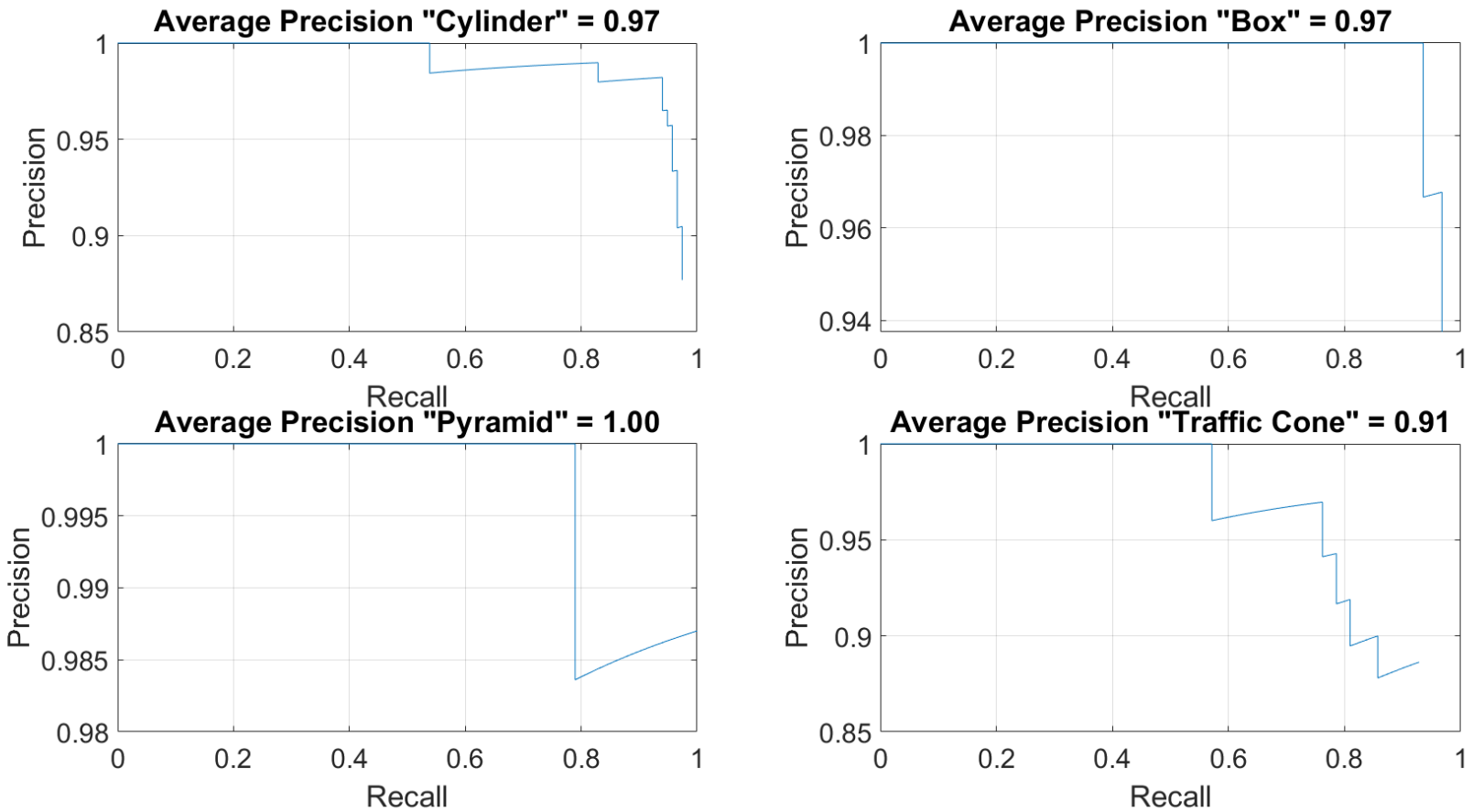


Figure: Precision Plot of Test Batch

Training & Testing Performace

Parameter	Worth
Average Detection Time	59 ms
mean IoU	0.8157
Training Accuracy	99.08%
Testing Accuracy	96.25%

Table: Performance Parameters of Training & Testing

Pie-Chart

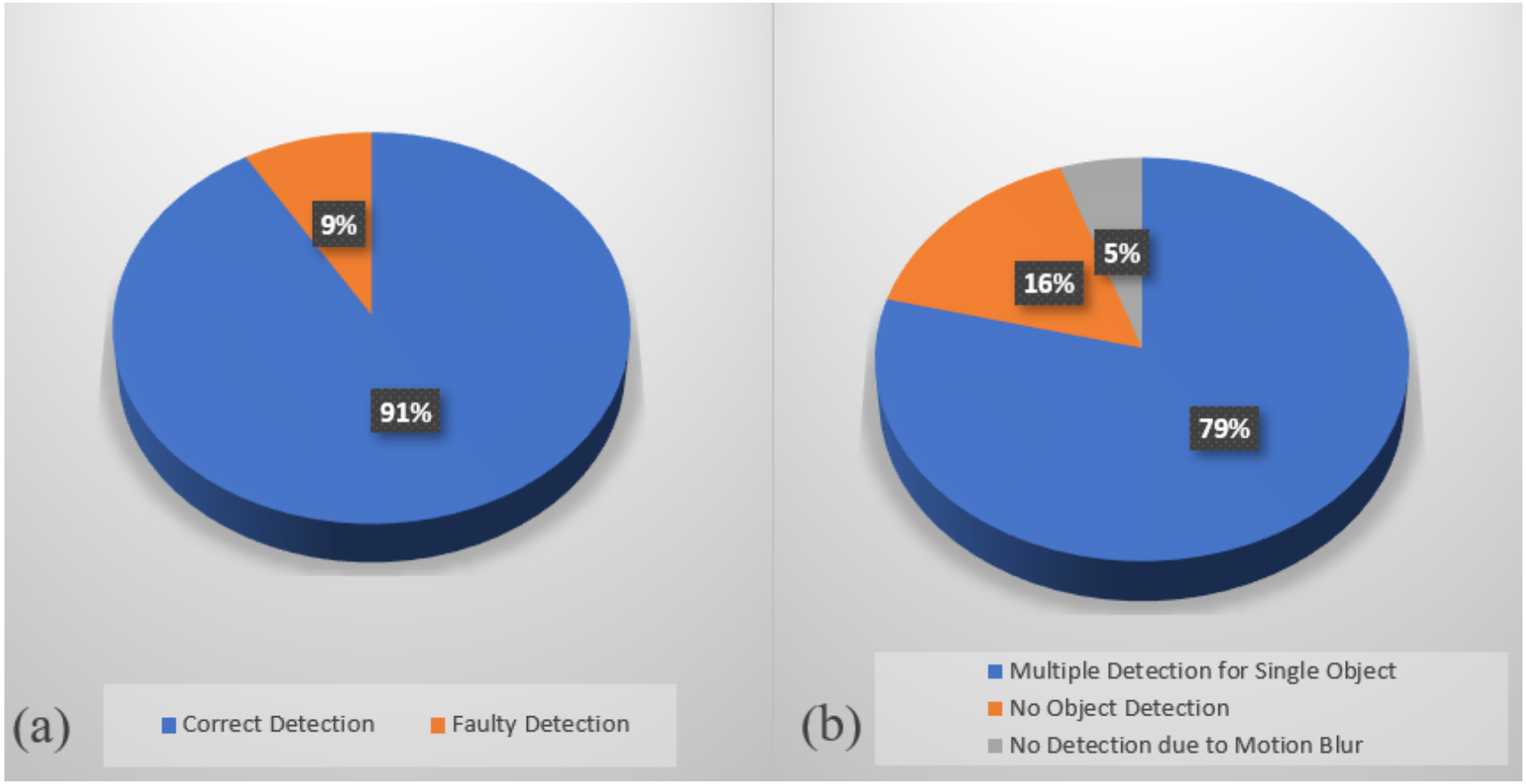


Figure: Pie Chart of Test Results

Object Detection Results

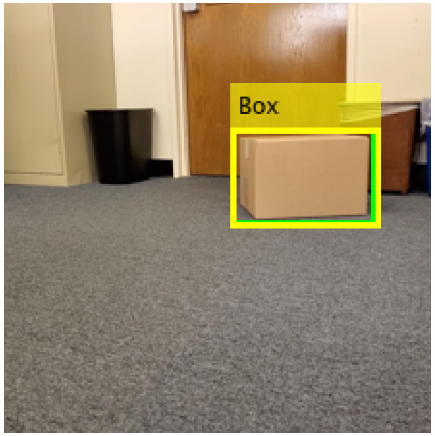


Figure: a

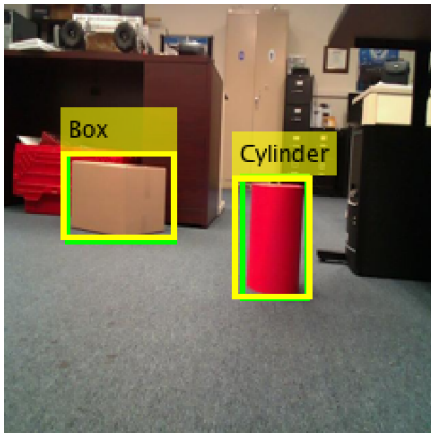


Figure: c

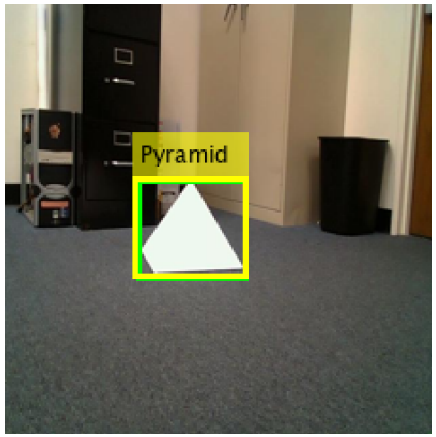


Figure: e

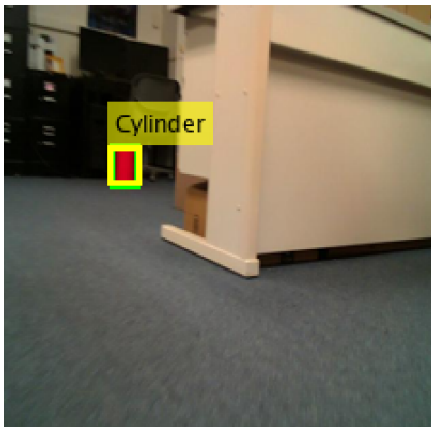


Figure: b

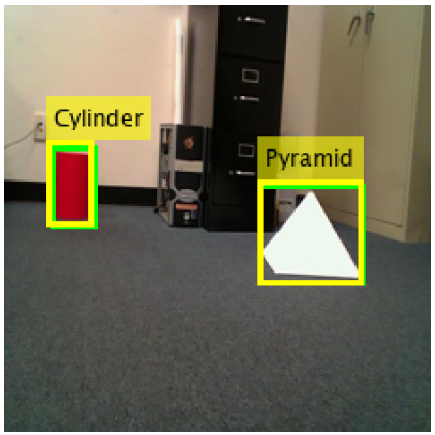


Figure: d

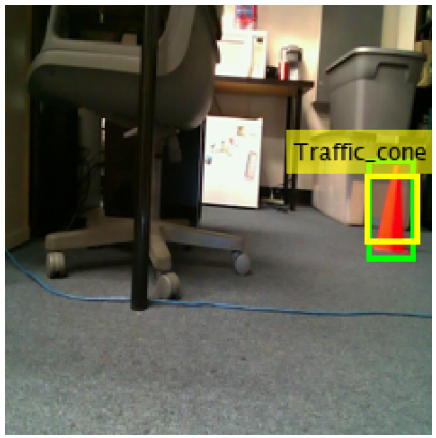


Figure: f

Faulty Detections

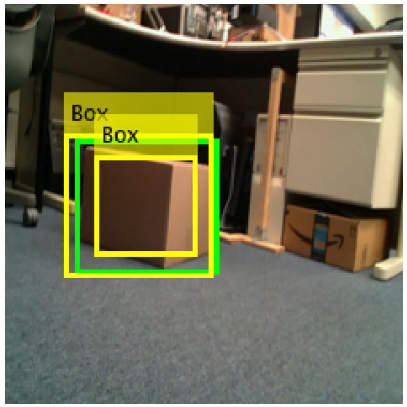


Figure: Multiple Detection for Single Object

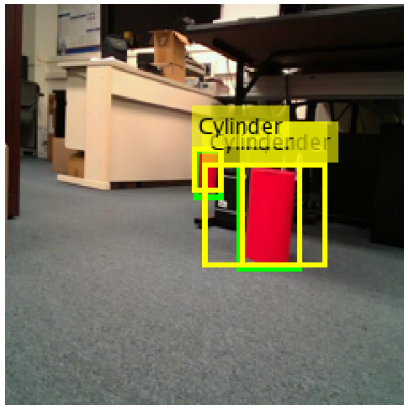


Figure: Multiple Detection for Single Object

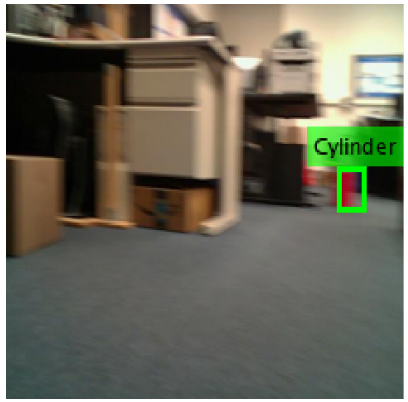


Figure: No Detection due to Motion Blur

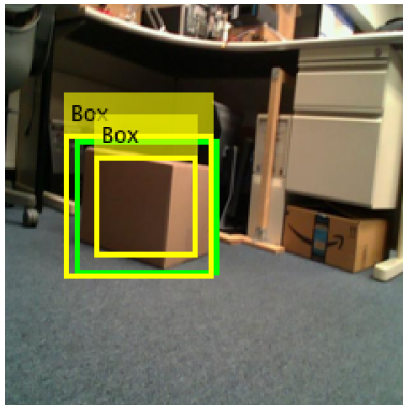


Figure: Multiple Detection for Single Object

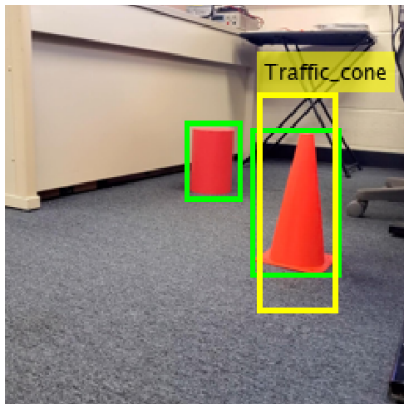


Figure: No Detection

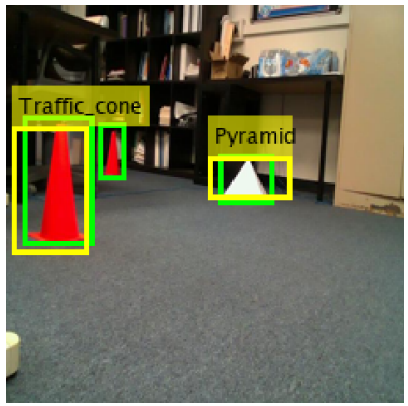


Figure: No Detection

Table of Contents

1. Motivation & Background
2. YOLO-v3 Architecture
3. Supervised Learning
4. Hyper-parameters Tuning
5. YOLO Algorithm
6. Training Results
7. Optical Sensor & Calibration
8. Sensor Fusion
9. Simulation Results
10. Experimental Results
11. Conclusion & Future Work

Single Pin-hole Camera Model

$$\begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = K \times \begin{bmatrix} R & t \end{bmatrix} \times \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix} K = \begin{bmatrix} f_x & 0 & 0 \\ s & f_y & 0 \\ c_x & c_y & 1 \end{bmatrix}$$

Where, K is a intrinsic matrix, R and t are rotation and translation matrices, (f_x, f_y) is a focal length in pixel, (c_x, c_y) is optical center coordinates in pixel, and s is skew coefficient.

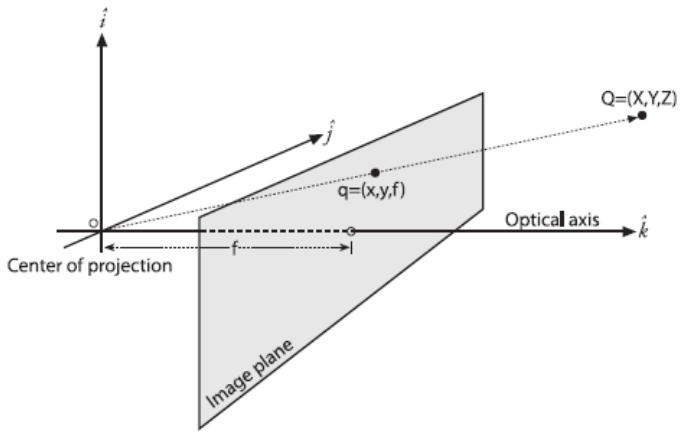


Figure: Single Pin-hole Camera Model [9]

[9] Gary Bradski and Adrian Kaehler. *Learning OpenCV: Computer Vision in C++ with the OpenCV Library*. 2nd. O'Reilly Media, Inc., 2013. ISBN: 1449314651

Stereo Camera

1:Undistortion

- Two types of distortion in camera: Radial and Tangential
- Light rays at the edge of a lens bend much more compare to rays close to the lens optical center which results in radial distortion
- Tangential distortion occurs when the lens plane is not completely parallel with the image plane

2:Rectification

- Images from two camera sensors are rotated and twisted to make the difference between their y coordinates to zero
- It is required when two cameras are not aligned and their image plane are not co-planner

3:Correspondence Formation

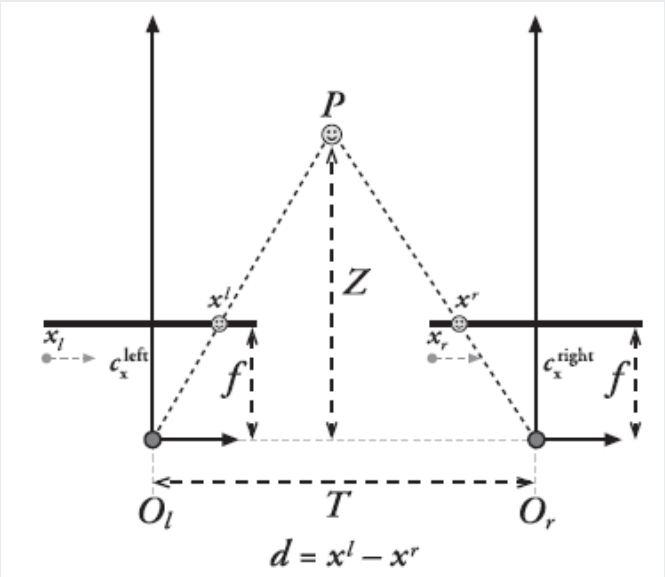
- A search algorithm is used that finds similar features in both images
- Final outcome is disparity map which is difference between x pixel coordinate of two image

4:Triangulation

- Depth information is derived by use of disparity and distance between two cameras
- Depth is inversely proportional to the disparity

$$Z = \frac{f \cdot T}{x^l - x^r}$$

Where, Z is a depth of object, f is a focal length, T is a distance between two cameras, and x^l and x^r are the projection of object P on the left and right camera image plane respectively



Optical sensor

Sony IMX 179

- Non-distortion lens with 75^o horizontal field of view
- Max. 30 fps in all pixel scan mode
- 8 MP high definition resolution
- Image Resolutions available: 3264 X 2448, 3200 X 2400, 1024 X 768, 1600 X 1200

Intel Realsense L515 LiDAR

- A laser exposure time is less than 100 ns per depth point
- A number of depth points per second is 9.2 M at 640 X 480 and 23.6 M at 1024 X 768 image resolution
- Depth accuracy is 5 mm to 14 mm

Standard Calibration Error

Description	Stereo Camera	Lidar
X Error	0.08 mm	0.10 mm
Y Error	-0.09 mm	-0.04 mm
Z Error	-0.03 mm	-0.19 mm

Table: Measurement Error in Intrinsic Calibration

Description	Camera to Lidar	Lidar to Camera
Transformation Error	29.5 mm	-29.42 mm

Table: Transformation Error in Extrinsic Calibration

Real-time Object Detection Results and Position Estimation

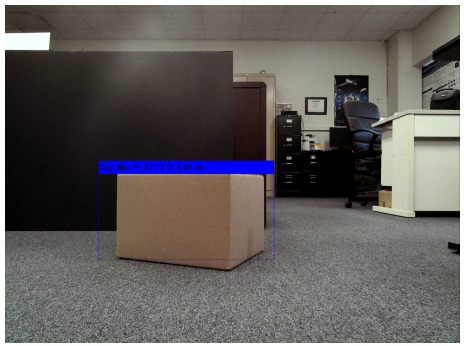


Figure: a

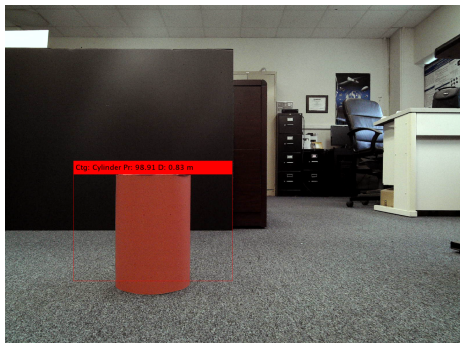


Figure: c

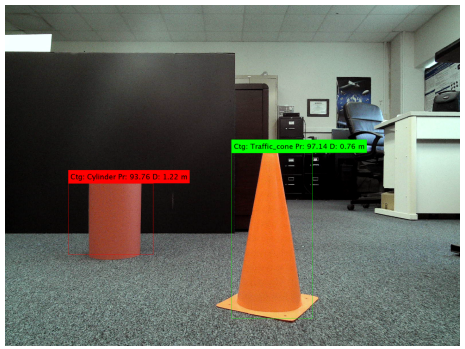


Figure: e

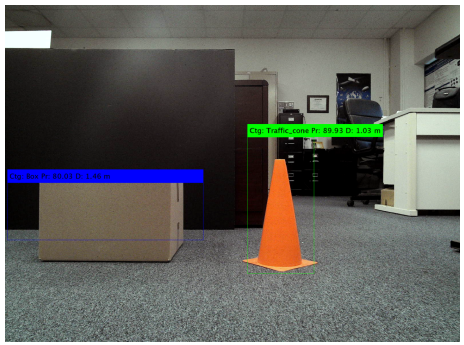


Figure: b

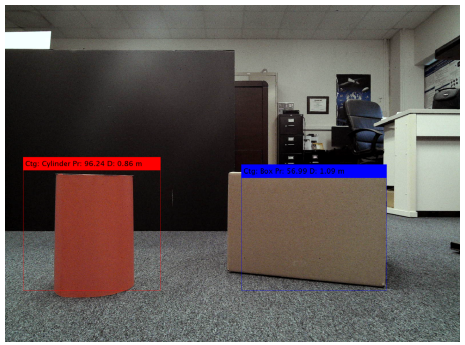


Figure: d

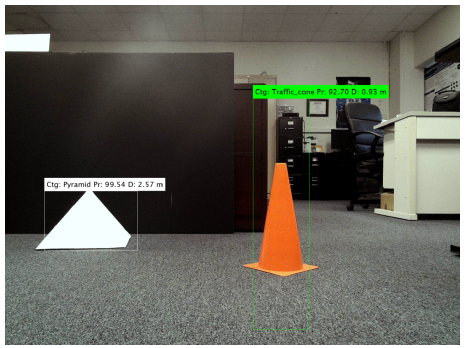


Figure: f

Table of Contents

1. Motivation & Background
2. YOLO-v3 Architecture
3. Supervised Learning
4. Hyper-parameters Tuning
5. YOLO Algorithm
6. Training Results
7. Optical Sensor & Calibration
8. Sensor Fusion
9. Simulation Results
10. Experimental Results
11. Conclusion & Future Work

Linear Kalman Filtering

System & Measurement Model

$$x_k = F_{k-1}x_{k-1} + G_{k-1}u_{k-1} + w_{k-1}$$
$$z_k = H_k x_k + v_k$$

Process Covariance

$$Q_k = \begin{bmatrix} q_{xx} & 0 & 0 \\ 0 & q_{yy} & 0 \\ 0 & 0 & q_{zz} \end{bmatrix}$$

Measurement Covariance

$$R_k = \begin{bmatrix} r_{xx} & 0 & 0 \\ 0 & r_{yy} & 0 \\ 0 & 0 & r_{zz} \end{bmatrix}$$

State Transition Metric

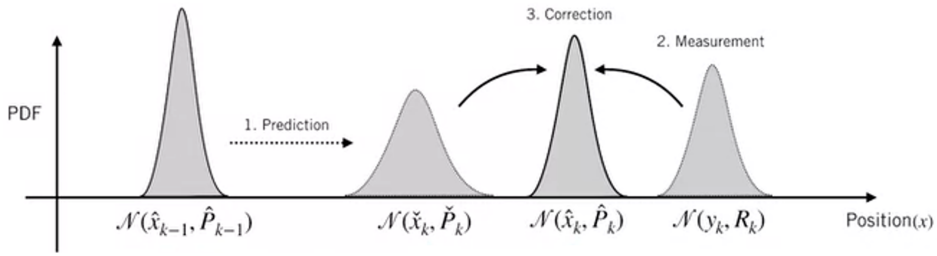
Constant Acceleration Model

$$F_{k-1} = \begin{bmatrix} 1 & T & \frac{1}{2}T^2 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & T & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & T & \frac{1}{2}T^2 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & T & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & T & \frac{1}{2}T^2 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & T \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

Measurement Metric

$$H_k = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \end{bmatrix}$$

Linear Kalman Filtering



Prediction

The state estimation of previous time stamp x_k is substituted in dynamic model to propagate the state forward as $x_{k+1|k}$.

$$x_{k+1/k} = F_k x_{k/k} + G_{k/k} u_k$$
$$P_{k+1/k} = F_k P_{k/k} F_k^T + Q_k$$

Measurement & Correction

The predicted states are corrected by using measurements through feedback.

$$K_{k+1} = P_{k+1/k} H_{k+1}^T \left(H_{k+1} P_{k+1/k} H_{k+1}^T + R_{k+1} \right)^{-1}$$
$$x_{k+1/k+1} = x_{k+1/k} + K_{k+1} \left(\tilde{z}_{k+1} - H_{k+1} x_{k+1/k} \right)$$
$$P_{k+1/k+1} = \left(1 - k_{k+1} H_{k+1} \right) p_{k+1/k}$$

Multi-sensor Data Fusion

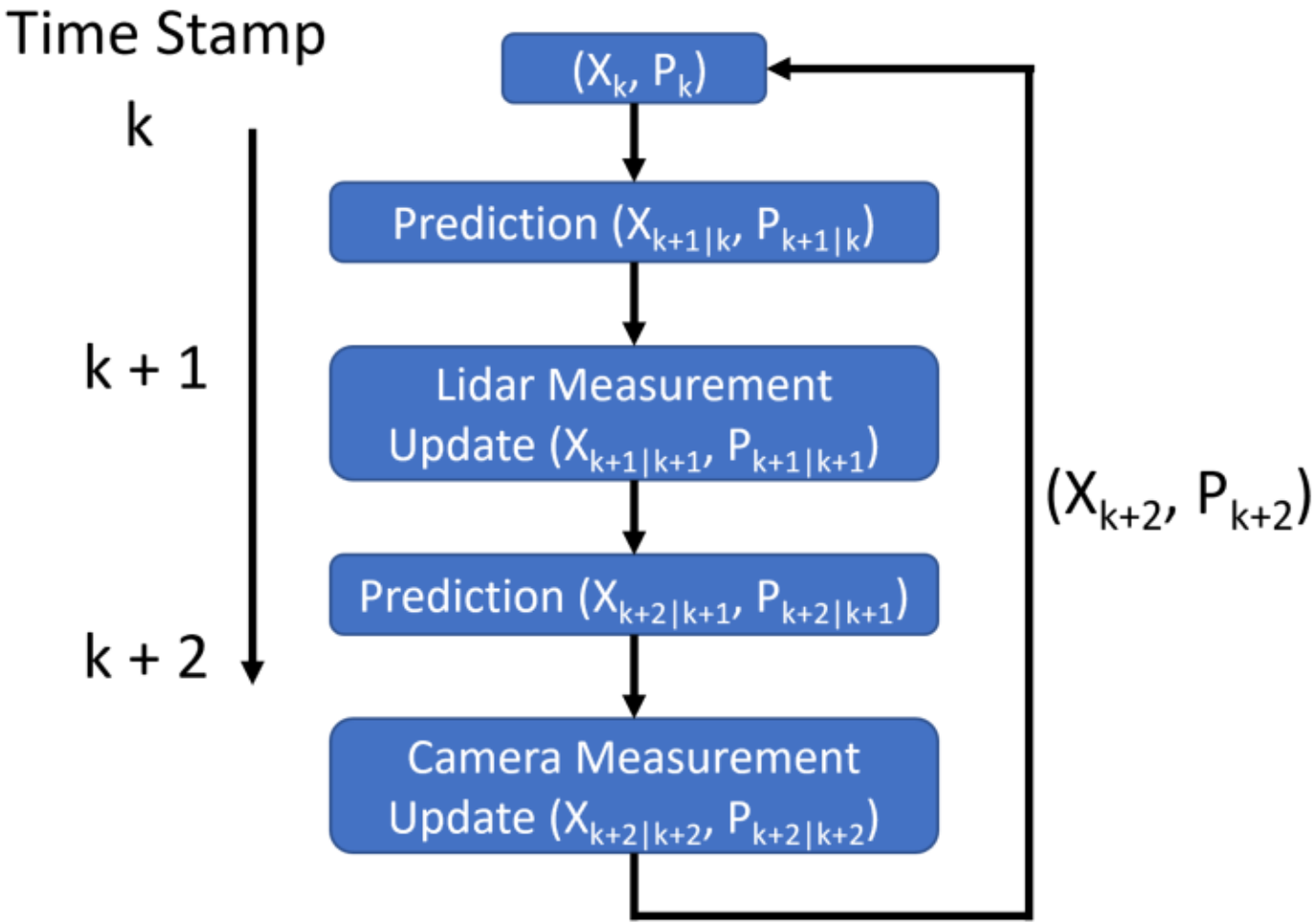


Figure: Flowchart of Multi-sensor Data Fusion for Asynchronous Case

Table of Contents

1. Motivation & Background
2. YOLO-v3 Architecture
3. Supervised Learning
4. Hyper-parameters Tuning
5. YOLO Algorithm
6. Training Results
7. Optical Sensor & Calibration
8. Sensor Fusion
9. Simulation Results
10. Experimental Results
11. Conclusion & Future Work

Driving Scene

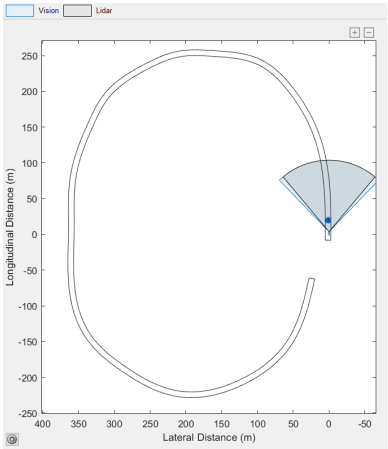


Figure: Bird's Eye View of Scene

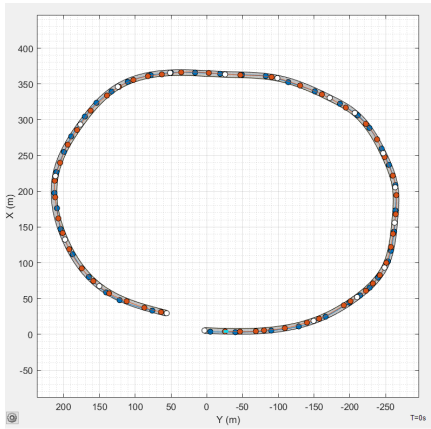


Figure: Way points Definition

Scene Parameter	**Worth**
Ego Vehicle Speed	30 m/s (67 mph)
Object Vehicle Speed	30 m/s (67 mph)
Camera Azimuthal Limits	[-43 43] (deg)
Camera Elevation Limits	[-90 90] (deg)
Lidar Azimuthal Limits	[-40 40] (deg)
Lidar Elevation Limits	[-90 90] (deg)

Table: Driving Scene Parameters

Sensor Co-variance & Measurement Error

Sensor Co-variance

$$R_{LiDAR} = \begin{bmatrix} 0.2102 & 0 & 0 \\ 0 & 0.0093 & 0 \\ 0 & 0 & 0.0001 \end{bmatrix}$$

$$R_{Camera} = \begin{bmatrix} 1.5336 & 0 & 0 \\ 0 & 1.3365 & 0 \\ 0 & 0 & 0.0001 \end{bmatrix}$$

Sensor Measurement Error

Description	**Camera**	**Lidar**	**Estimation**
X Error	0.95 m	0.29 m	0.40 m
Y Error	0.20 m	0.033 m	0.14 m
X Error	0.0001 m	0.0001 m	0.00001 m
Overall Error	0.97 m	0.29 m	0.42 m

Simulation

Video

Position Plot

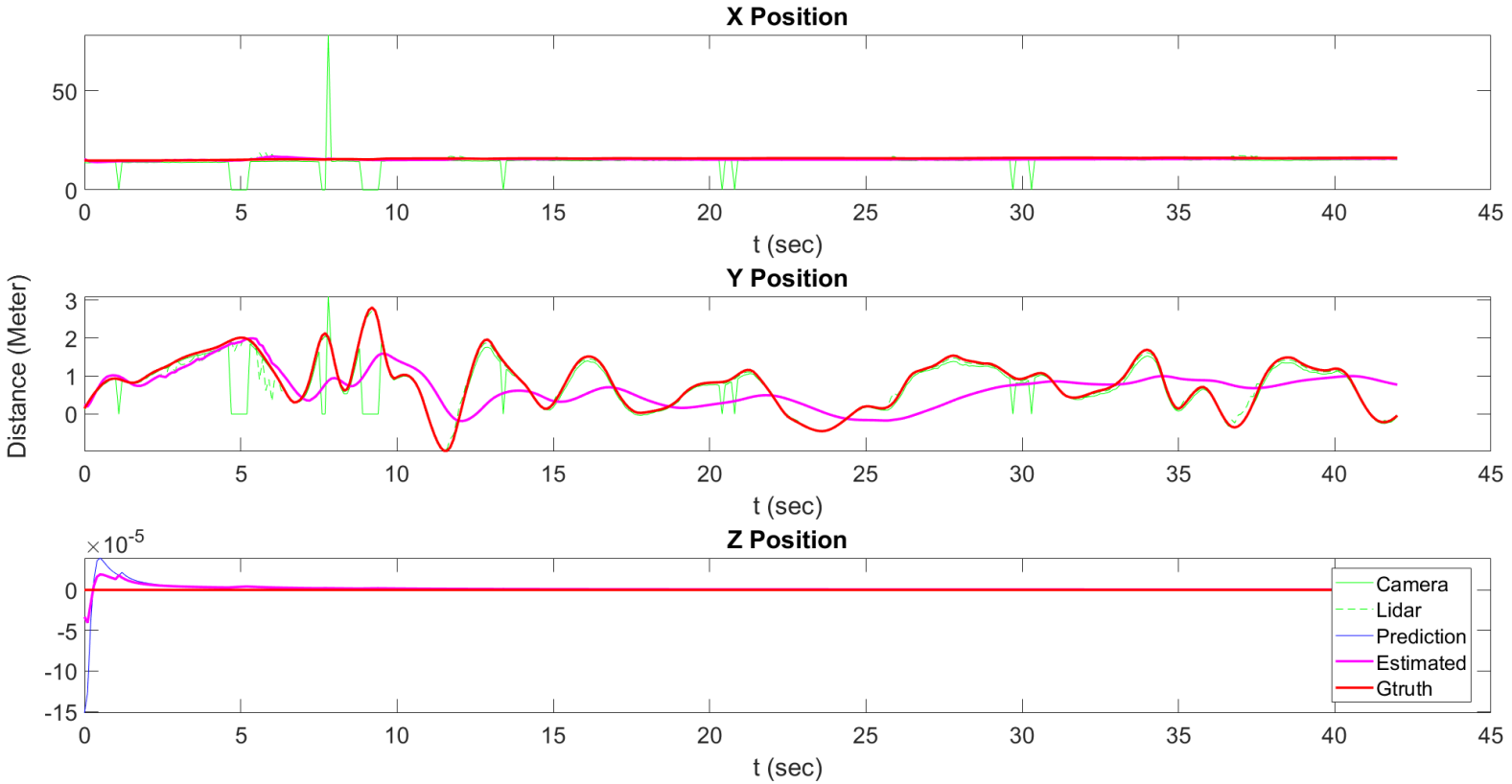


Figure: Position plot in Simulation

Velocity Plot

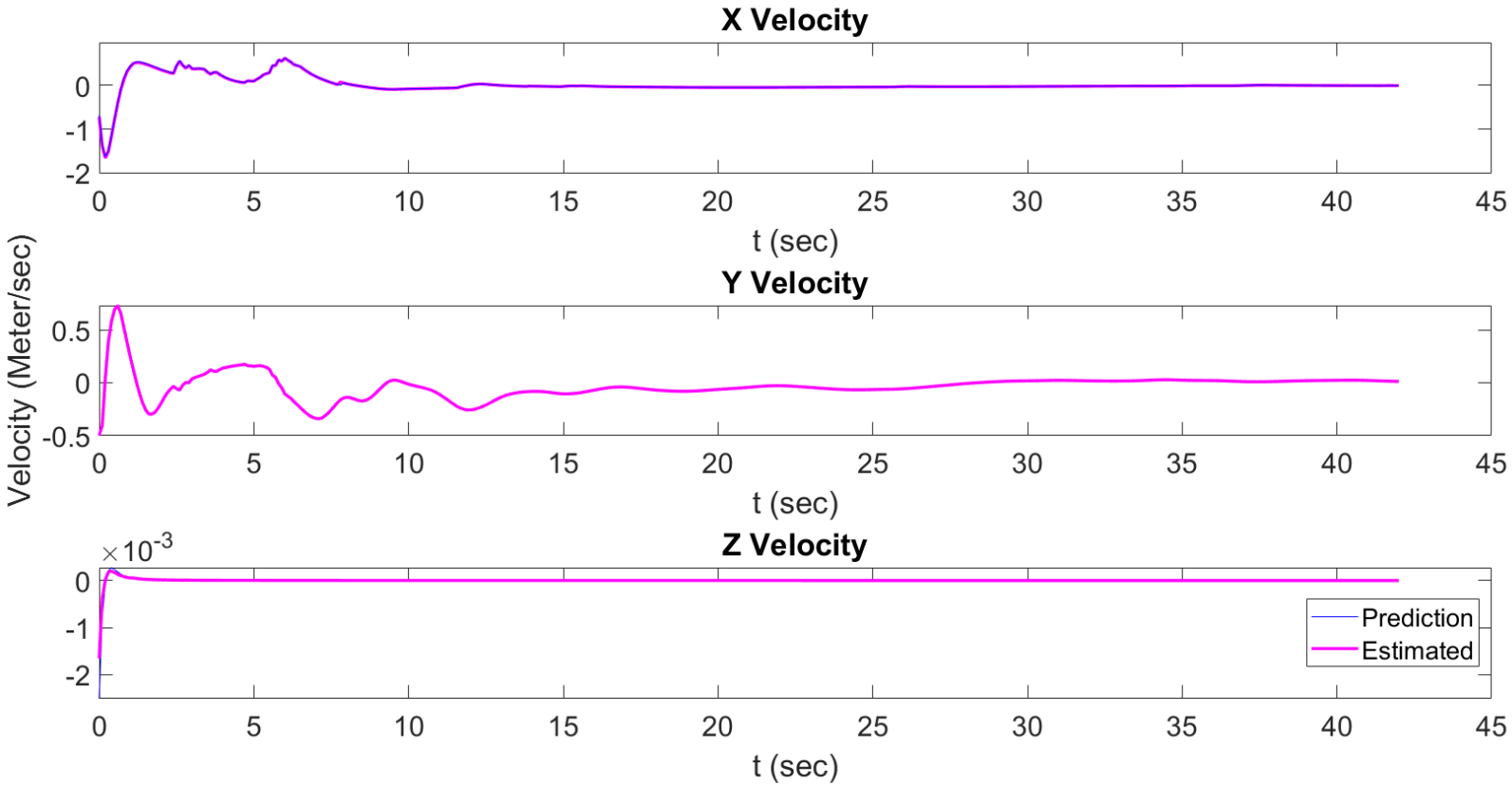


Figure: Velocity plot in Simulation

Estimation Error Plot

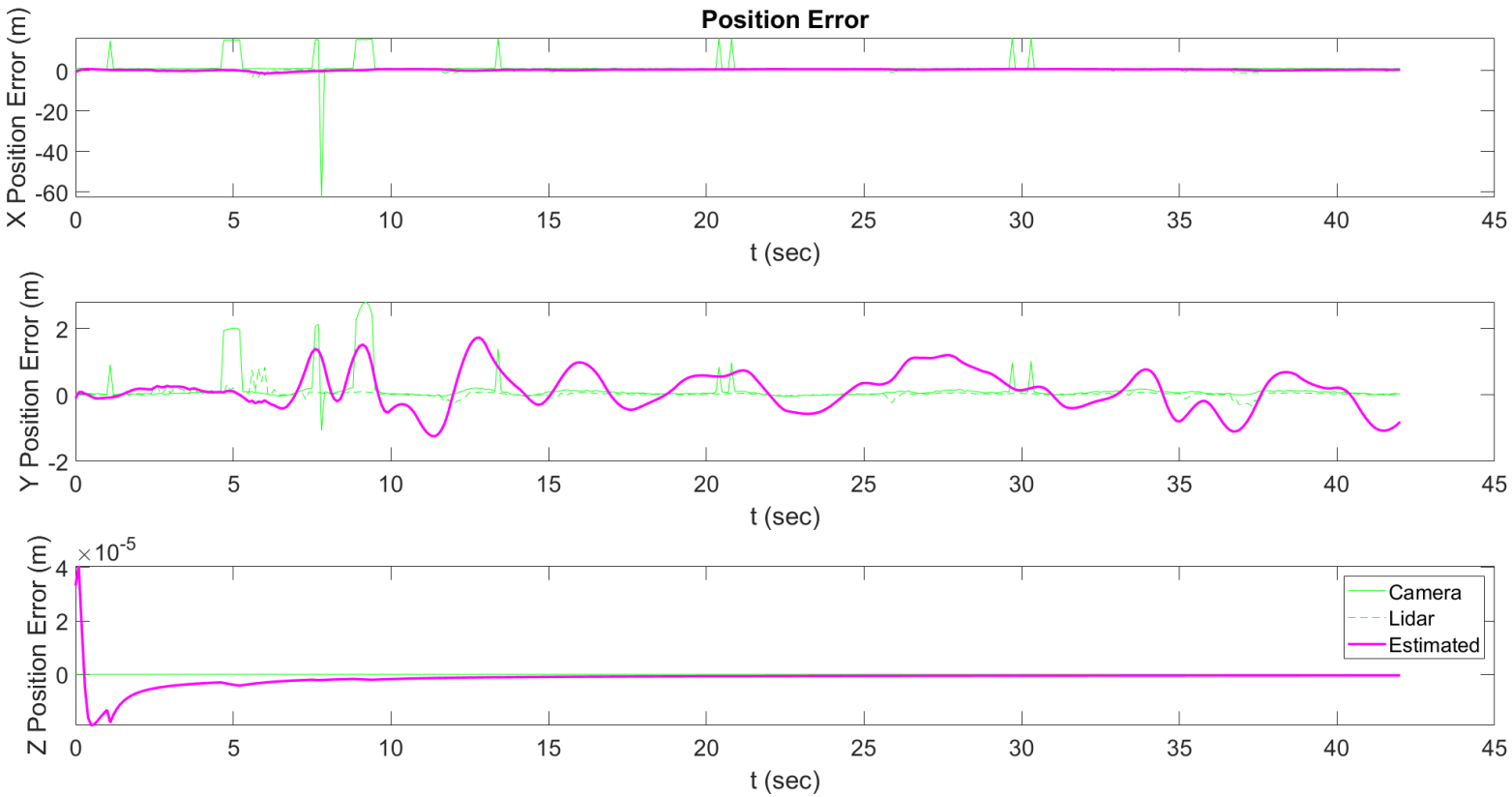
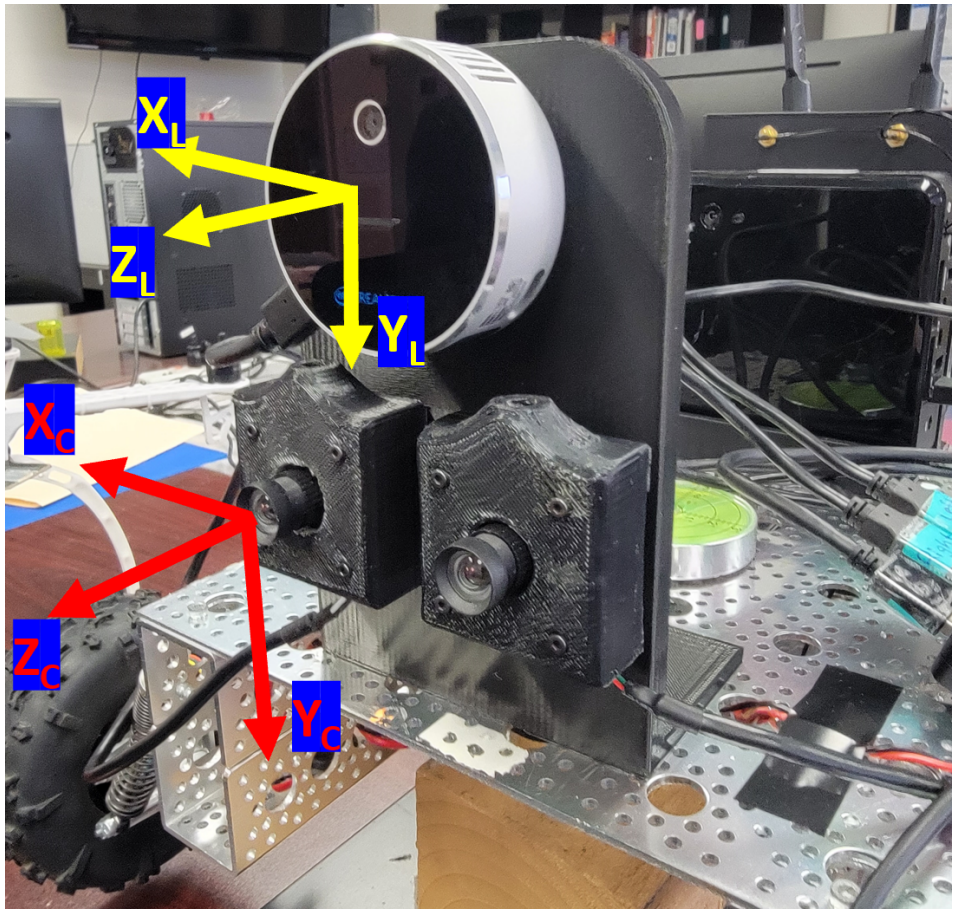


Figure: Sensor Measurement and Estimation Error Plot

Table of Contents

1. Motivation & Background
2. YOLO-v3 Architecture
3. Supervised Learning
4. Hyper-parameters Tuning
5. YOLO Algorithm
6. Training Results
7. Optical Sensor & Calibration
8. Sensor Fusion
9. Simulation Results
10. Experimental Results
11. Conclusion & Future Work

Co-ordinate Axis



Sensor Co-variance

$$R_{LiDAR} = \begin{bmatrix} 5.84e-05 & 0 & 0 \\ 0 & 2.73e-04 & 0 \\ 0 & 0 & 3.917e-06 \end{bmatrix}$$

$$R_{Camera} = \begin{bmatrix} 2.59e-05 & 0 & 0 \\ 0 & 9.76e-06 & 0 \\ 0 & 0 & 4.46e-04 \end{bmatrix}$$

Real-time Position Estimation

Video

Position Plot "Box"

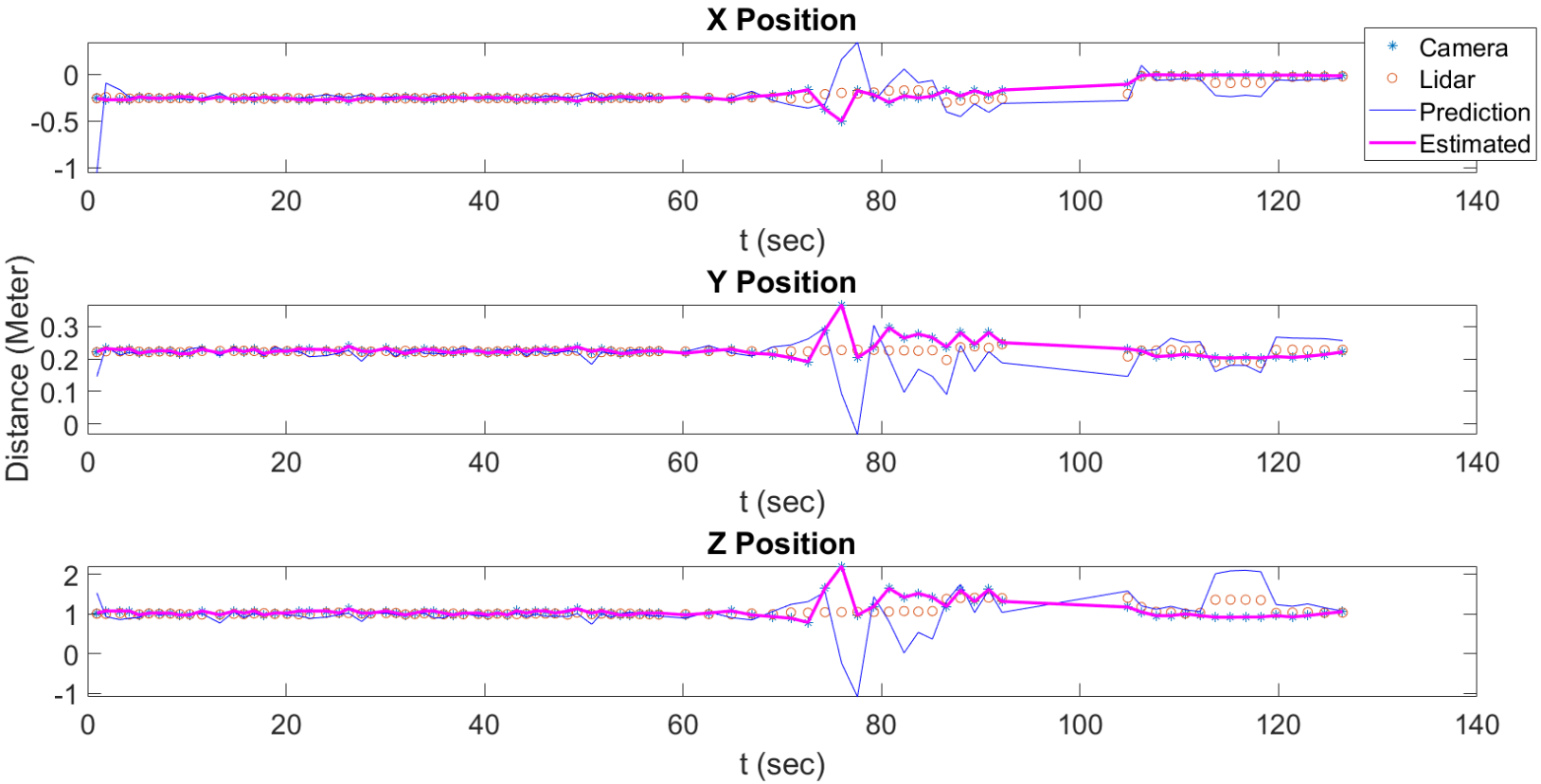


Figure: Position plot of an Object "Box"

Velocity Plot "Box"

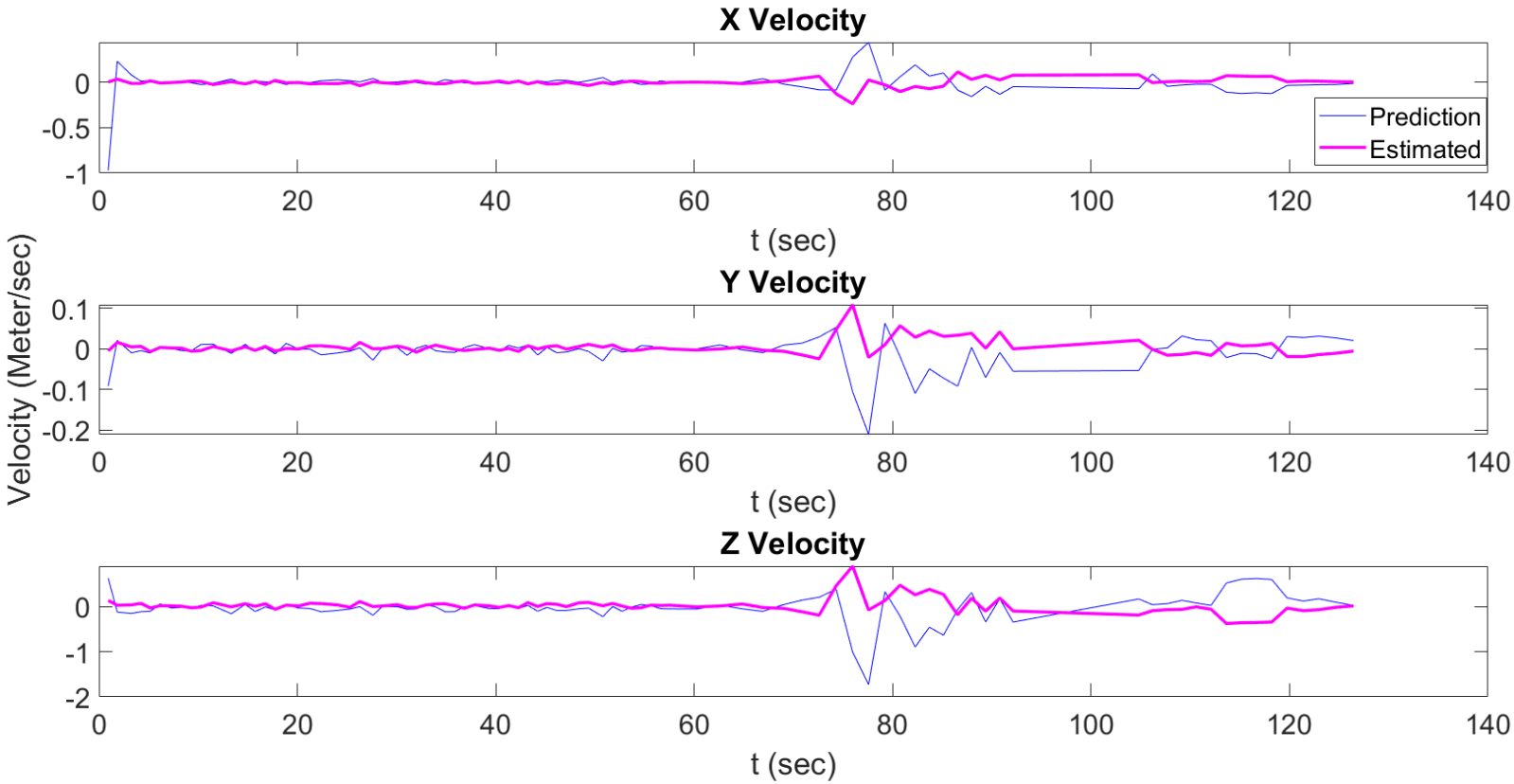


Figure: Velocity plot of an Object "Box"

Position Plot "Traffic Cone"

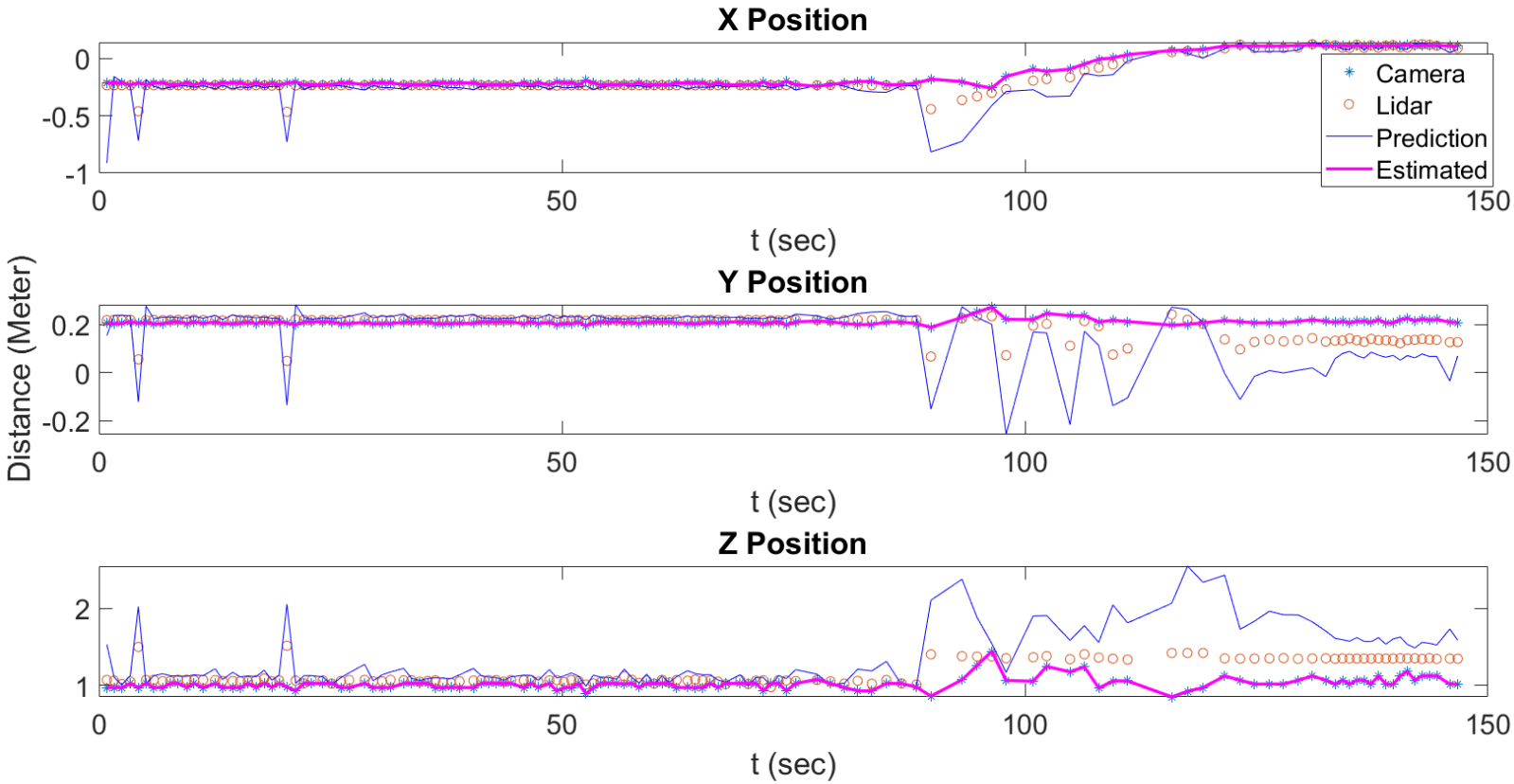


Figure: Position plot of an Object "Traffic Cone"

Velocity Plot "Traffic Cone"

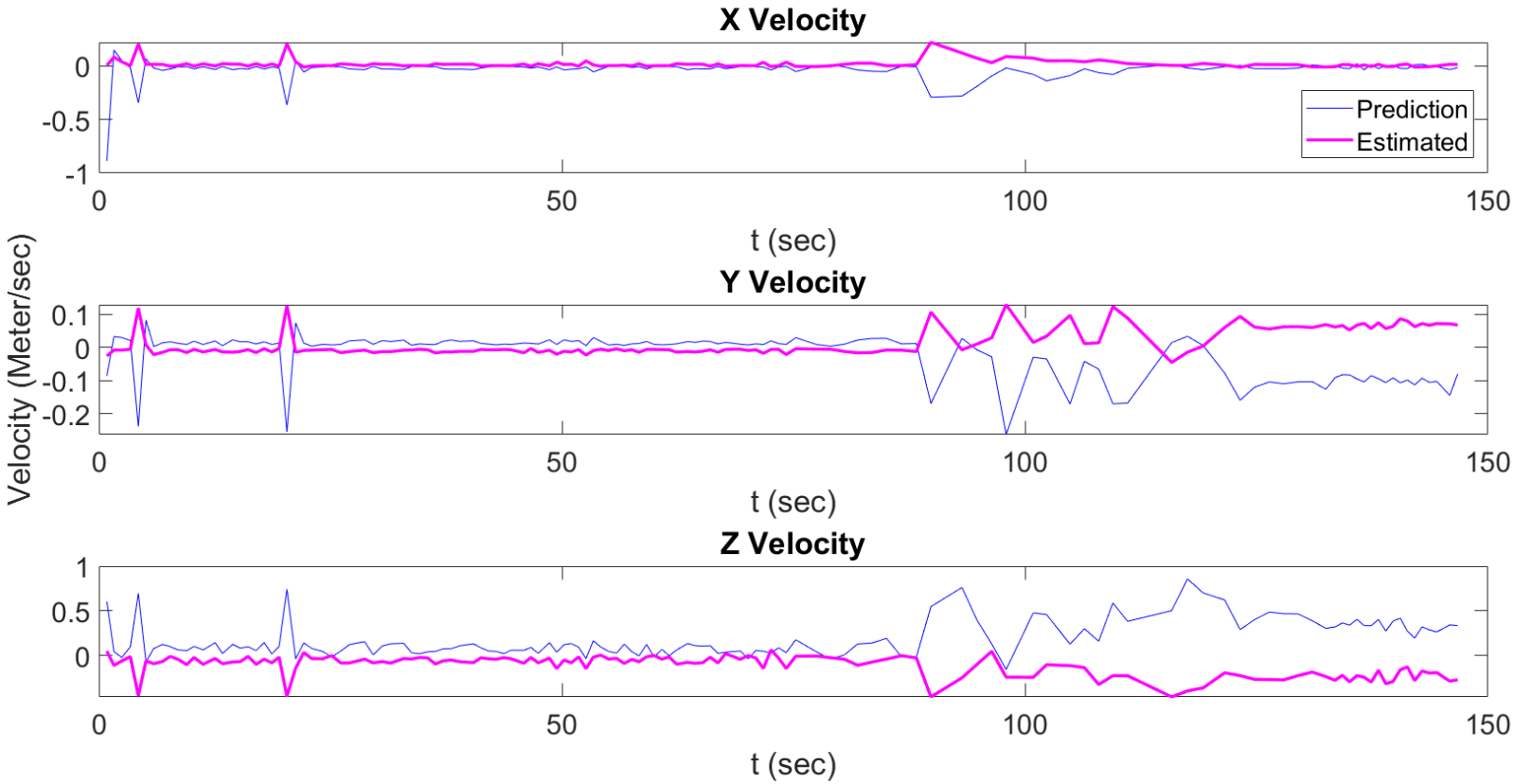


Figure: Velocity plot of an Object "Traffic Cone"

Table of Contents

1. Motivation & Background
2. YOLO-v3 Architecture
3. Supervised Learning
4. Hyper-parameters Tuning
5. YOLO Algorithm
6. Training Results
7. Optical Sensor & Calibration
8. Sensor Fusion
9. Simulation Results
10. Experimental Results
11. Conclusion & Future Work

Conclusion

- It was shown in this work that the YOLO-v3 unified algorithm performs better and faster than most of the modern object detection neural networks.
- The training and testing accuracy of 99% and 96% respectively were achieved.
- An intrinsic and extrinsic calibration of Lidar and stereo camera was carried out to compute the measurement and transformation error of camera and Lidar data.
- Finally, a sensor fusion algorithm was implemented by using a linear Kalman filter to fuse the data coming from Lidar and the stereo camera.

Future Work

- Develop a more sophisticated multi-object tracking algorithm; for instance JPDA (Joint Probabilistic Data Association).
- Further this study can be extended to implement the collision avoidance, and SLAM (Simultaneous Localization and Mapping).
- The neural network can be trained to detect more complex objects that might be encountered in the real world.
- The YOLO-v3 network is further tuned to improve test accuracy.
- By use of the LiDAR with a stronger laser that can use in ambient conditions, this research work can be extended to autonomous driving or advanced driving assistance system.
- The Lidar and camera calibrations can be automated and made more accurate to improve the overall system accuracy in the future.